

Programação Concorrente e Distribuída

Introdução

Glêdson Elias
gledson@ci.ufpb.br

1



Introdução

- Programa Sequencial
  - Não mais que uma operação acontece em um dado momento
  - É decorrente da forma como o controle é passado em sequência entre comandos
  - Cada comando deve ser concluído antes do próximo comando iniciar
- Programa Concorrente
  - É hábil a realizar mais que uma operação em um dado momento

2

Introdução

- Concorrência
  - Concorrência não é uma tarefa trivial
  - Torna a programação mais difícil
  - Dificulta a realização dos testes
  - Reduz a efetividade dos testes

Programas sequenciais são razoavelmente entendíveis. Complexidade é proporcional ao tamanho

Programas concorrentes em geral não compartilham tais características

Introdução

- Motivação

Escrever programas corretos é difícil

Escrever programas concorrentes corretos é muito mais difícil

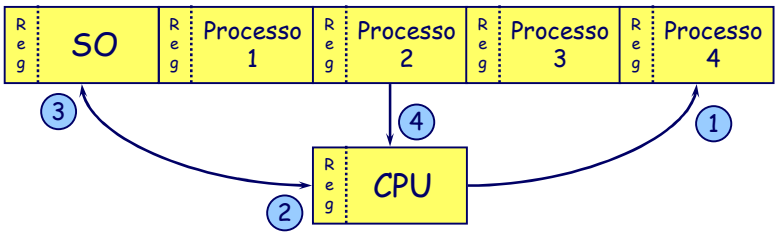
Porque desenvolver programas concorrentes?

Introdução

- Motivação
  - Busca de maior desempenho
  - Representação de aspectos concorrentes do mundo real
  - Utilização das arquiteturas de processadores concorrentes

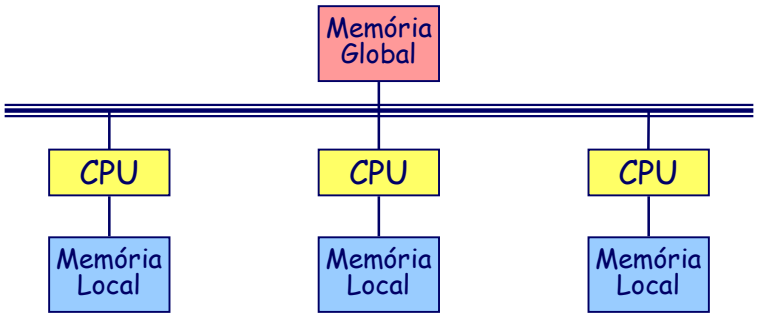
Introdução

- Arquiteturas de Sistemas
  - Sistema Multitarefa
    - Permite vários processos compartilharem o processador
    - Compartilhamento baseado na divisão do tempo
    - Concorrência de processamento e entrada/saída



Introdução

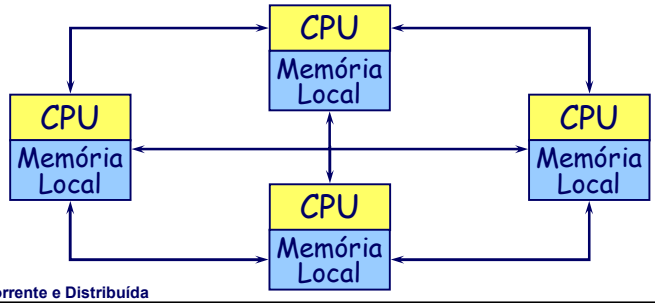
- Arquiteturas de Sistemas
  - Sistema Multiprocessador
    - Aloca processos nos vários processadores
    - Também acontece o compartilhamento de processadores



7

Introdução

- Arquiteturas de Sistemas
  - Sistemas Distribuídos
    - Consiste de vários computadores que operam independentemente mas que se comunicam
    - Provê maior desempenho agregado e incrementa a confiabilidade



8

Introdução

- Tipos de Concorrência
  - Concorrência Física (Paralelismo)
    - Cada processo é executada em um processador dedicado
  - Concorrência Lógica
    - Processador é chaveado de um processo para outro
    - Causa a impressão que processos executam simultaneamente

Introdução

- Programas e Processos
  - Processo Sequencial
    - É um conjunto totalmente ordenado de passos
      - Dado quaisquer dois passos é sempre possível indicar qual é o primeiro
    - Cada passo sendo uma mudança de estado em algum componente do sistema de computação

Introdução

- Programas e Processos
  - Programa Sequencial
    - Especifica as possíveis mudanças de estado de um processo sequencial
    - Estrutura de controle do programa determina a ordem de execução
    - Ações são ordenadas pelas regras de sequenciação da linguagem de programação adotada
      - Define como o controle é passado de comando a comando

Introdução

- Programas e Processos
  - Programa Concorrente
    - Especifica as possíveis mudanças de estado de dois ou mais processos sequenciais
    - Nenhuma ordem é implicitamente definida entre as mudanças de estados dos processos
    - Processos são ditos executar concorrentemente
    - Processos podem até mesmo executar simultaneamente

Introdução

- Problemas com Concorrência
  - Determinismo x Não-Determinismo
    - Programas sequências corretos são determinísticos
      - Seguem uma sequência de passos que pode ser completamente reproduzida em múltiplas execuções com os mesmos dados de entrada
    - Determinismo torna possível verificar e validar programas via testes

Introdução

- Problemas com Concorrência
  - Determinismo x Não-Determinismo
    - Um programa concorrente é não-determinístico
      - Unidades de execução prosseguem em uma ordem não predefinida
    - Não é possível predizer a sequência de passos que é realizada ou o resultado final
    - Múltiplas execuções do programa concorrente com os mesmos dados de entrada podem gerar resultados diferentes
    - Sequência de passos e resultado final dependem do escalonamento interno

Introdução

- Problemas com Concorrência
  - Determinismo x Não-Determinismo
    - Um programa concorrente incorreto . . .
      - Pode ter o comportamento esperado a grande maioria das vezes
      - Pode de forma intermitente e irreprodutível desviar-se do comportamento normal
    - Erros de programação concorrente são muito difíceis de diagnosticar
    - Não-determinismo é um ponto chave da programação concorrente
    - Torna bastante difícil a programação

Introdução

- Problemas com Concorrência
  - Dependência de Velocidade
    - Um programa sequencial é independente da velocidade
      - Corretude não depende da velocidade de execução
    - Resultado de um programa concorrente é dependente da velocidade
      - Depende da velocidade relativa com que os processos sequenciais constituintes são executados
      - Pequenas flutuações randômicas na velocidade do processador e dispositivos de entrada/saída podem levar ao não-determinismo

Introdução

- Problemas com Concorrência
  - Dependência de Velocidade
    - Quando os resultados são dependentes da velocidade diz-se que existe uma condição de corrida (race condition)
      - Resultados podem ser imprevisíveis

```
P: s := "ABCDE"
Q: s := "FGHIJ"
```

`s = "ABCDE"` `s = "FGHIJ"` `s = "ABHIE"`

• • •

Introdução

- Problemas com Concorrência
  - Dependência de Velocidade
    - Requer a adoção do conceito de atomicidade
      - Uma variável é atômica quando é inspecionada e atualizada sem qualquer interrupção no processamento
    - Atomicidade não resolve o problema geral

```
P: i := i + 1
Q: i := 2 * i
```

PQ `i = 2` QP `i = 1`

```
LD reg, i
INC reg
ST i, reg
```

```
LD reg, i
ML reg, 2
ST i, reg
```

`i = 0`

Introdução

- Problemas com Concorrência
  - Deadlock
    - Dois ou mais processos encontram-se impossibilitados de fazer qualquer progresso no processamento
    - Gerado em função das mútuas demandas incompatíveis por recursos
    - Pode ocorrer se e somente se as seguintes condições são satisfeitas
      - Exclusão Mútua
      - Aquisição Incremental
      - Não Preempção
      - Espera Circular

Introdução

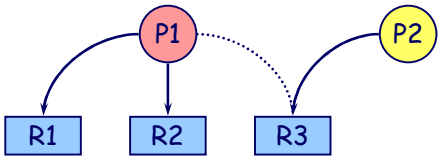
- Problemas com Concorrência
  - Deadlock
    - Exclusão Mútua
      - Processos possuem acesso exclusivo aos recursos



Enquanto um processo escreve dados no arquivo, não faz sentido permitir outro processo usar o arquivo ao mesmo tempo

Introdução

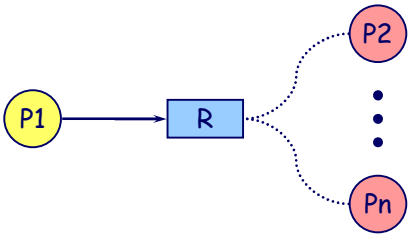
- Problemas com Concorrência
  - Deadlock
    - Aquisição Incremental
      - Processo mantém recursos previamente adquiridos enquanto aguarda por outro recurso



Enquanto um processo espera pelo arquivo, não faz sentido liberar o outro arquivo previamente adquirido para copiar dados

Introdução

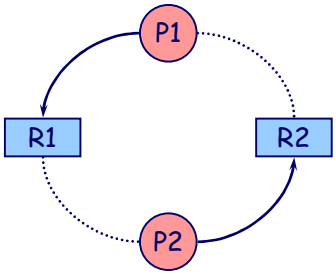
- Problemas com Concorrência
  - Deadlock
    - Não Preempção
      - Recursos não podem ser removidos de um processo até que sejam voluntariamente liberados



Um arquivo alocado a um dado processo não pode ser liberado até que operações de atualização sejam concluídas

Introdução

- Problemas com Concorrência
  - Deadlock
    - Espera Circular
      - Pode existir um ciclo de recursos e processos no qual cada processo está aguardando recursos que são mantidos pelo próximo processo no ciclo



Processo P1 mantém um arquivo, que é requerido pelo processo P2, que, por sua vez, mantém outro arquivo, que é requerido por P1

Introdução

- Problemas com Concorrência
  - Starvation
    - Processo é indefinidamente impedido de executar em função de escalonamento injusto
      - Safety
        - Coisas ruins não acontecem
      - Liveness
        - Coisas boas eventualmente acontecem
        - Processo fará algum progresso em um dado intervalo de tempo
    - Somente pode ser evitado se o sistema é livre de deadlock e adota um escalonamento justo

Introdução

- Problemas com Concorrência
  - Starvation
    - Escalonamento Justo
      - Processo é colocado em uma fila associada ao recurso requisitado
      - Se recurso torna-se disponível, um processo na fila eventualmente ganha o acesso ao recurso
    - Escalonamento Injusto
      - Acesso preferencial a processos de alta prioridade
      - Pode retardar indefinidamente um processo de baixa prioridade

Introdução

- Interações entre Processos
  - Programas concorrentes diferem dos programas sequenciais não apenas pela presença de composição concorrente
  - Também diferem pela presença de operações que causam interações entre processos

Introdução

- Interações entre Processos
  - Processos Independentes
    - Processos B e C são independentes se . . .
      - Nenhum passo de B afeta o comportamento de C
      - Nenhum passo de C afeta o comportamento de B
      - Qualquer passo de B pode ser executado antes, ao mesmo tempo ou depois de qualquer passo de C
      - Qualquer passo de C pode ser executado antes, ao mesmo tempo ou depois de qualquer passo de B

Introdução

- Interações entre Processos
  - Processos Independentes
    - Composição concorrente de processos independentes é determinística
      - Permite que servidores multi-acesso possam processar múltiplas requisições baseado no modelo multitarefa ou multiprogramação
      - Não requer que os usuários tenham cuidados especiais para tratar a concorrência

Introdução

- Interações entre Processos
  - Processos Competidores
    - Processos B e C são competidores se . . .
      - Ambos requerem acesso exclusivo a algum recurso compartilhado em um ou vários passos
    - Passos de B e C que requerem acesso exclusivo são denominados regiões críticas

B: B_1; B_2; B_3
C: C_1; C_2; C_3

. . . ; B_2; . . . ; C_2; . . .

. . . ; C_2; . . . ; B_2; . . .

Propriedade safety somente pode ser obtida se as sessões críticas nunca são executadas com concorrência física ou lógica

Introdução

- Interações entre Processos
  - Processos Competidores
    - Resultados dos processos competidores são em geral não determinísticos
      - Resultados das sessões críticas podem depender do estado do recurso compartilhado
      - Sessões críticas podem mudar o estado do recurso compartilhado
    - Resultados dependem da velocidade relativa dos processos competidores

Introdução

- Interações entre Processos
  - Processos Competidores
    - Não determinismo limitado (bounded)
      - Resultado final não é previsível mas pertence a um conjunto fixo e conhecido de possíveis resultados aceitáveis

```
P: i := i + 1
Q: i := 2 * i
```

```
i = 2
```

```
i = 1
```


Introdução

- Interações entre Processos
  - Processos Comunicantes
    - Processos B e C são comunicantes se . . .
      - Algum passo de B (B_p) produz dados que algum passo de C (C_p) consome
        - B_p deve concluir antes de iniciar C_p
      - Algum passo de C (C_p) produz dados que algum passo de B (B_p) consome
        - C_p deve concluir antes de iniciar B_p
    - Processos B e C são intercomunicantes se existe comunicação em ambas direções

Exercício I

```
a = 1
b = 2
c = 3
```

a := b + 1	c := a + b	b := c*2
LD rx, b INC rx ST a, rx	LD rx, a LD ry, b ADD rx, ry ST c, rx	LD rx, c MUL rx, 2 ST b, rx

Quais os possíveis valores finais das variáveis a, b e c?

Exercício II

- Dado um conjunto de processos $P = \{p_1, p_2, ..., p_n\}$ e um conjunto de arquivos $A = \{a_1, a_2, ..., a_n\}$
  - Considere que cada processo p_i pode bloquear, acessar e liberar até dois arquivos a_j e a_k
    - Para $1 \leq i < n$ $j = i$ $k = i+1$
    - Para $i = n$ $j = i$ $k = 1$
  - Quando acessa os dois arquivos, o processo deve primeiro acessar um dos arquivos, e, mantendo-o bloqueado, acessar o outro arquivo.

O sistema é livre de deadlock?

Existe alguma situação específica em que ocorre deadlock?

Caso afirmativo, proponha um padrão de acesso que evite deadlock.